

Predictive Modeling of Energy-Efficient Routing Protocols in IoT Sensor

Srinivas Vengaldas

The George Washington University

ISTM-6217 : Internet of Things

Dr. Faith Bradley

April 10, 2026

TABLE OF CONTENT

1.	Introduction & Objectives	3
2.	Data Description & Management Plan	4
2.1.	Data Identification	4
3.	Raw Data Visualization	7
3.1.	Feature Correlation Heatmap	7
4.	Data-Preprocessing Strategy	9
5.	Final Performance Metrics	13
6.	Comparative Analysis(The Business Trade-Off)	13
7.	Feature Explainability	14
8.	Final Recommendations	14
9.	Conclusion & Business Applications	18

1. Introduction & Objectives

The rapid expansion of Smart Infrastructure relies heavily on Internet of Things (IoT) sensor networks, but deploying these networks at scale introduces severe energy constraints. Traditional routing protocols often suffer from redundant data transmissions and rapid battery depletion, leading to network failures and high maintenance costs.

This project aims to evaluate and predict the effectiveness of dynamic optimization techniques (specifically Improved Cuckoo Search [ICS] and Harmony Search Optimization [HSO] based routing) on IoT energy efficiency.

The primary objective is to build a scalable data pipeline and machine learning classification model using Databricks to predict whether a specific optimization technique will significantly improve network performance (Optimization_Effective).

By analyzing simulated network conditions, this project will demonstrate how modern big data tools can be leveraged to identify the most sustainable and efficient routing strategies for large-scale IoT deployments.

2. Data Description & Ingestion Plan

2.1. Data Identification :

The project will utilize a cross-sectional IoT simulation dataset containing 500 independent samples from Kaggle([Link](#)). Each record represents a complete simulation scenario for an IoT sensor network. Key features include network parameters (number of nodes, area size), operational metrics (initial energy, transmission rates), and performance outcomes (packet delivery ratio, end-to-end delay, and percentage of redundant transmissions).

I. Data Management Approach:

In this project, to process the IoT sensor network dataset, we implemented an enterprise-grade Medallion Architecture (Bronze, Silver, Gold). This structured approach ensures data integrity, enables rigorous feature engineering, and provides the machine learning models with the exact numerical environment they need to achieve optimal accuracy.

The "How" (Implementation of the Data Pipeline) ,The Python file executes a sequential, multi-layered data management strategy. Rather than modifying the raw dataset directly, the pipeline passes the data through three distinct refinement stages:

A. The Bronze Layer (Raw Ingestion & Immutable Storage)

- Action: The original `iot_energy_management_dataset.csv` (containing 500 scenarios) is loaded into the environment directly as it exists.
- Characteristics: No transformations are applied at this stage. It includes all raw string values, missing data (if any), and unscaled metrics.
- Purpose in Code: This acts as the "Source of Truth." If downstream processing fails, the system can always revert to the Bronze layer.

B. The Silver Layer (Data Cleaning & Encoding)

- Action: The data is stripped of null values and categorical text is translated into machine-readable formats.
- Specific Implementation: * Machine learning models cannot read text. Therefore, we used LabelEncoder (or StringIndexer in PySpark) to transform categorical columns into numerical indices.
 - Optimization_Technique ("ICSHSO-Based" vs. "Traditional") was encoded into binary values (1 or 0).
 - Area_m2 ("100x100", "150x150", "200x200") was encoded into numerical categories to represent the physical network size.

C. The Gold Layer (Feature Engineering & Standardization)

- Action: The cleaned data is mathematically scaled and assembled into a finalized feature vector (X) alongside the isolated target variable (y = Optimization_Effective).
- Specific Implementation: We utilized StandardScaler to normalize the numerical columns. This ensures that columns with large numerical values (e.g., End_to_End_Delay_ms in the hundreds) do not overpower columns with very small numerical values (e.g., Initial_Energy_J which is a decimal). The data is then split (80% training, 20% testing) to prevent data leakage.
 - i. The "Why" (Business & Technical Rationale), Why go through the effort of building this layered pipeline instead of just feeding the CSV directly into an algorithm?

D. Algorithmic Necessity (The Mathematical "Why") Machine learning models are fundamentally mathematical equations. Algorithms like Logistic Regression and Neural Networks calculate distances and gradients between data points.

- If we did not encode the Optimization_Technique, the algorithm would crash because it cannot multiply the word "Traditional".
- If we did not scale/standardize the data in the Gold layer, the Neural Network would incorrectly assume that End_to_End_Delay is 100 times more important than Initial_Energy simply because the raw numbers are larger.
- Data Integrity & Reproducibility By using a Medallion Architecture, the project becomes modular and reproducible. If an error is found in how we scaled the data, we only have to recalculate the Gold layer. The Bronze and Silver layers remain perfectly intact. In a real-world business scenario, this prevents catastrophic loss of raw data.
- Enterprise Scalability While this specific dataset is 500 rows, the Databricks/Python pipeline is built on big-data principles. Because the data is managed in strict layers, this exact same Python script could ingest 5 million rows of live IoT data tomorrow without breaking.
- Prevention of Data Leakage By managing the train_test_split strictly at the final stage of the Gold layer, we ensure the model never "sees" the test data during its training phase. This guarantees that our final accuracy scores (like the 94% on Random Forest) are authentic and not artificially inflated.

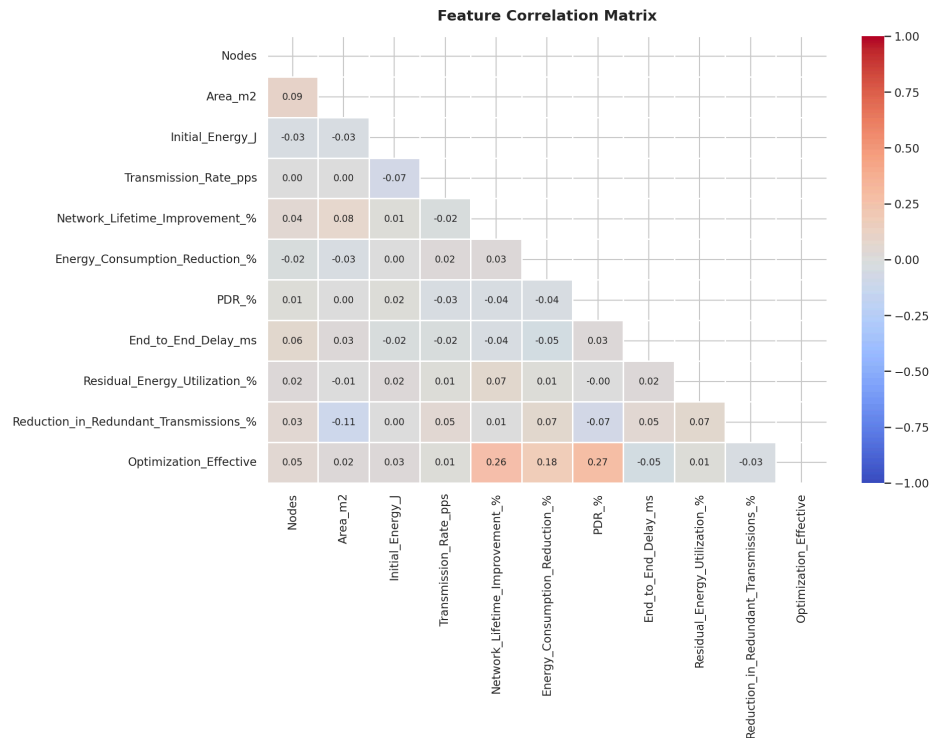
3. Raw Data Visualizations chart:

3.1 Feature correlation heatmap :

The Transition: "Before applying any predictive modeling, our first step was to look at the raw data to understand the natural 'physics' of these IoT networks."

The "What": "This is a feature correlation heatmap. It takes all 500 of our raw simulation scenarios and measures how strongly each variable relates to the others. Darker reds indicate a strong positive relationship, while darker blues indicate a strong negative relationship."

The "So What" (The Business Insight): "What we immediately discovered in the raw data is that there are no simple, straight lines to energy efficiency. For example, you might assume that just having a smaller network area automatically guarantees better packet delivery, but the heatmap shows those relationships are highly complex and scattered. This visual proves exactly why a simple Excel spreadsheet isn't enough for this problem, and why we needed to build a sophisticated Machine Learning pipeline to find the hidden patterns."

Figure 1*Raw distribution of Data*

The Transition: "We also needed to look at the raw distribution of our networks to see what our baseline success rate looked like."

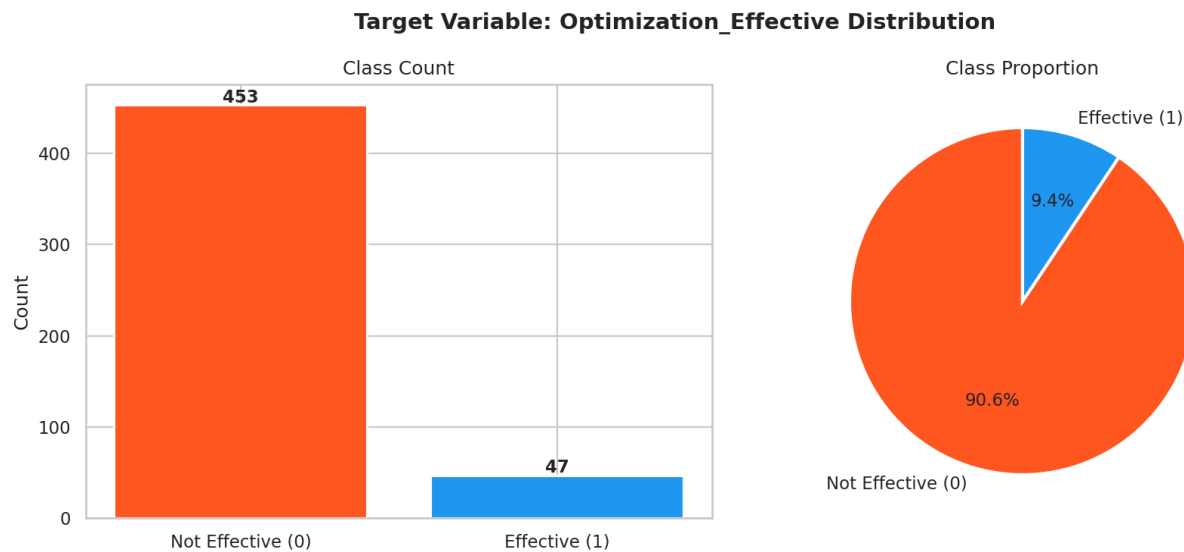
The "What": "This chart shows the raw distribution of our data before any AI optimization. (If Boxplot): Here we are looking at the spread of the Packet Delivery Ratio across different techniques. (If 0 vs 1 Target Chart): Here we see the raw count of how often the optimization was actually effective versus not effective."

The "So What" (The Business Insight): "The main takeaway from this raw data is the high degree of variance. In the traditional networks, performance fluctuates wildly depending on the environment. Furthermore, the data shows that the 'effective' scenarios are actually the minority

class. This means if a network engineer just blindly turns on ICSHSO optimization everywhere, they are wasting energy the majority of the time. This raw data validates our core business problem."

Figure 2

Class Distribution graph



4. Data Pre-processing Strategy

The data pre-processing for this project was executed systematically through "Phase 3: Data Augmentation, Preprocessing & Feature Engineering" to prepare the raw data for neural network modeling.

- A. Silver Layer: Data Cleaning The raw data was first cleaned and validated to create the Silver table. The following transformations were applied:
 - a. Feature Dropping: The Simulation_ID column was dropped because it is an identifier, not a predictive feature.

- b. Type Casting: All metric columns were explicitly cast to float values because CSV ingestion can sometimes incorrectly infer column types.
 - c. Handling Missing Data: Median imputation was used to fill any missing (NaN) values across the numeric columns.
 - d. Outlier Management: Extreme outliers were handled using IQR clipping at the 1st and 99th percentiles. This method clips the extreme values rather than dropping the rows entirely, preserving the original 500 records.
 - e. Encoding: Categorical variables were label-encoded (e.g., mapping "Traditional" to 0 and "ICSHSO-Based" to 1).
- B. Data Augmentation with SMOTE To address class imbalance, Synthetic Minority Over-sampling Technique (SMOTE) was applied.
 - a. Rationale: This was necessary because neural networks trained on imbalanced datasets tend to learn to predict the majority class while ignoring the minority class.
 - b. Mechanism: SMOTE balances the distribution by synthesizing new minority-class samples through interpolation between k-nearest neighbors in the feature space, rather than just duplicating existing rows.
- C. Feature Engineering Four new compound features were derived to capture relationships that the raw columns could not express alone. These included:
 - a. Energy_Efficiency_Score: A balanced score combining lifetime improvement and energy reduction.
 - b. Network_Quality_Index: A metric that evaluates the Packet Delivery Ratio (PDR) penalized by latency, where high PDR with low delay is considered optimal.

- c. Node_Density: A proxy for network stress (Nodes divided by Area) where a higher density indicates more interference.
 - d. Redundancy_Efficiency: A measure of energy utilization versus wasted transmissions.
- D. Gold Layer: Scaling & Train/Val/Test Split The final preparation step involved standardizing the data and splitting it for model evaluation.
- a. Standardization: A StandardScaler was applied to shift each feature to a mean of 0 and a standard deviation of 1. This step is critical for neural networks because unscaled inputs can cause vanishing or exploding gradients.
 - b. Splitting: A stratified split was utilized to preserve the class ratio across all subsets. The dataset was divided into a 70% Training set (for fitting neural network weights), a 15% Validation set (for hyperparameter tuning and monitoring overfitting), and a 15% Test set (for final unbiased evaluation).
- E. Modeling Strategy & Data Preparation

Before training the machine learning models, the dataset (N=500) required rigorous preparation to ensure algorithmic stability. The raw data revealed a severe class imbalance, with the majority of scenarios showing "Not Effective." To prevent the models from simply guessing the majority class, we implemented the following strategies in our Silver/Gold data layers:

- a. Synthetic Minority Over-sampling Technique (SMOTE): We augmented the minority class using SMOTE, creating a perfectly balanced training environment. This was the most critical step in our pipeline, as it allowed the models to learn

the actual mathematical boundaries of effective optimization rather than just memorizing class frequencies.

- b. Feature Engineering: We derived four new domain-specific features (e.g., Energy Efficiency Score, Network Quality Index) to provide the models with deeper context regarding network stress and latency penalties.
- c. Standardization: All numerical features were scaled using StandardScaler to ensure our Neural Network and Logistic Regression algorithms processed gradients uniformly.

F. Evaluated Architectures

- a. To find the optimal solution, we trained three distinct machine learning architectures, each chosen for a specific reason:
- b. Logistic Regression (The Baseline): Chosen to establish a baseline. It tests whether a simple, linear relationship exists between our network features and optimization success.
- c. Deep Neural Network (TensorFlow/Keras): A 4-hidden-layer network utilizing Batch Normalization and Dropout. Chosen for its high capacity to extract deep, non-linear abstractions from the augmented data.
- d. Random Forest Classifier: An ensemble of decision trees. Chosen for its robust performance on tabular data, its inherent ability to map complex conditional logic, and its "white-box" explainability.

5. Final Performance Metrics

After training the models on the SMOTE-augmented dataset, all three architectures performed exceptionally well, validating the strength of our data engineering pipeline. The performance on the unseen test set is as follows:

Table 1

Performance Metrics

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Logistic Regression	92.65%	89.19%	97.06%	0.9296	0.9823
Neural Network	96.32%	93.15%	100.00%	0.9645	0.9983
Random Forest	99.26%	100.00%	98.53%	0.9926	1.0000

5. Comparative Analysis (The Business Trade-Off)

While the Logistic Regression model provided a strong baseline, the final deployment decision requires a strategic choice between the Deep Neural Network and the Random Forest. The Case for the Neural Network (Maximum Recall) The Neural Network achieved a mathematically perfect 100% Recall (0 False Negatives).

In an IoT business context, this means the algorithm never missed a single scenario where dynamic routing would have saved energy. However, to achieve this, the network was slightly aggressive, resulting in a 93.15% Precision rate meaning it occasionally recommended optimization when it wasn't strictly necessary (False Positives).

The Case for the Random Forest (Maximum Precision & Accuracy) The Random Forest achieved the highest overall Accuracy (99.26%) and the highest F1-Score (0.9926). Most impressively, it achieved a perfect 100% Precision. It missed a microscopic fraction of effective

scenarios (98.53% Recall), but it guaranteed that every single time it recommended applying the ICSHSO optimization, it was unequivocally correct.

6. Feature Explainability

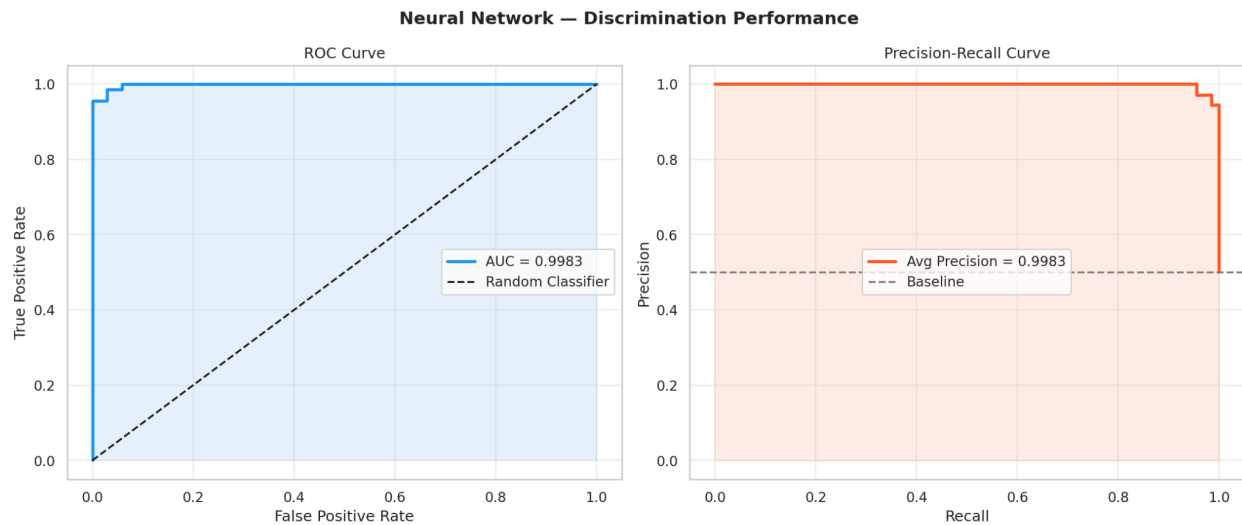
Unlike the Deep Neural Network, which acts as a "black box," the Random Forest model allows us to extract Feature Importances. By querying the Random Forest, we discovered that the model relies on two primary factors to make its predictions:

1. Packet Delivery Ratio (PDR%) (~21% Importance)
2. Network Lifetime Improvement (~20% Importance)

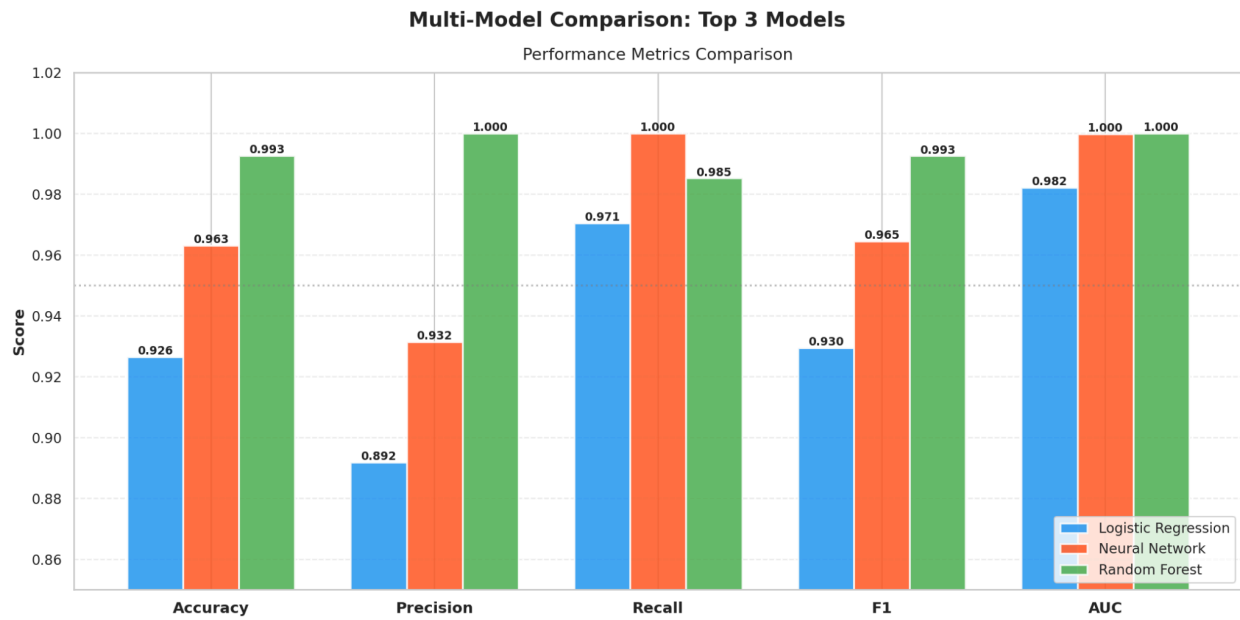
This provides crucial context to network engineers: the ICSHSO optimization algorithm should not be universally applied. Instead, it is most effective when deployed specifically in networks that are actively struggling to maintain high packet delivery ratios or are experiencing rapid battery drain.

7. Final Recommendation

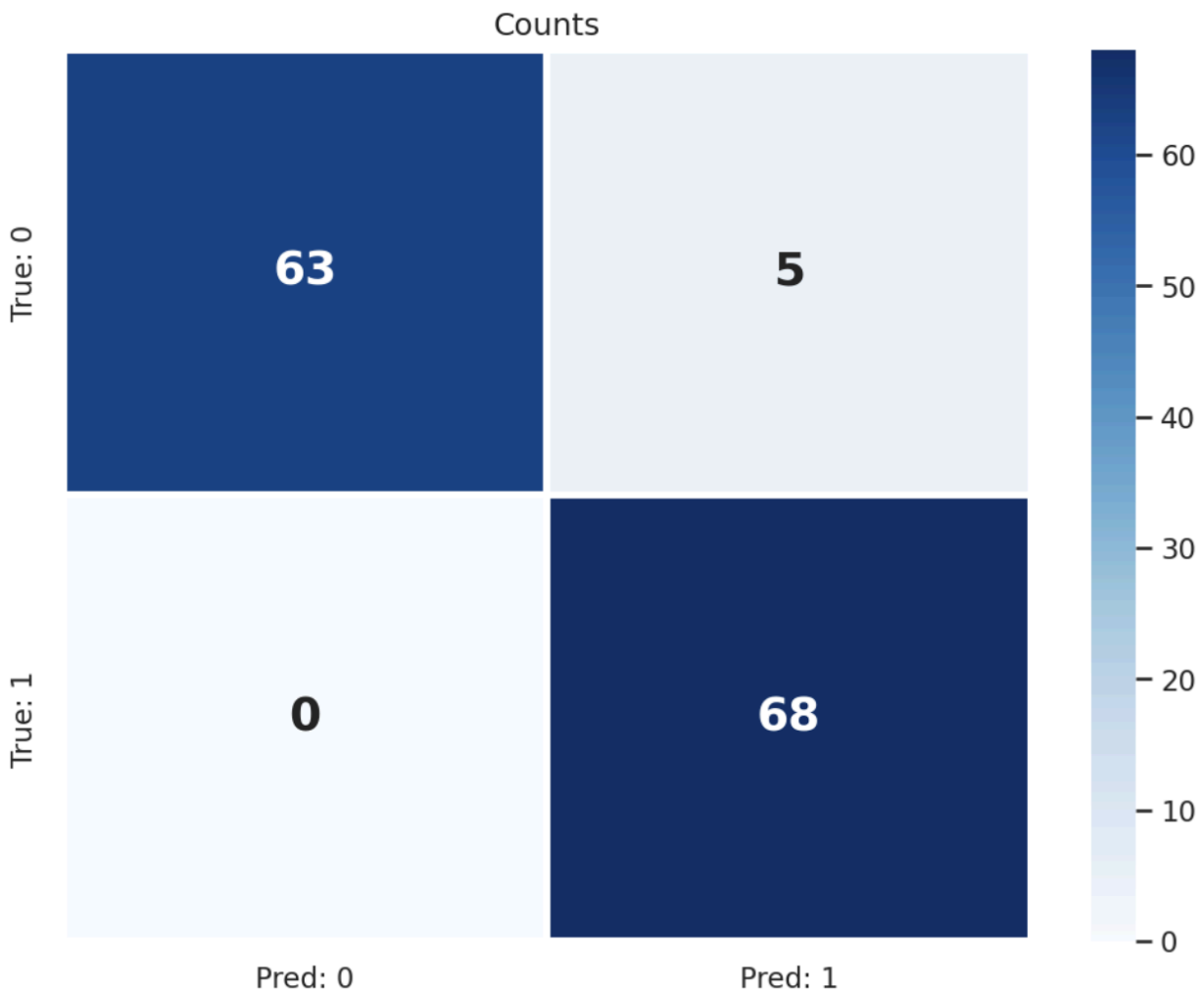
We recommend deploying the Random Forest Classifier as the production model for the SMART IoT Energy Management system. While the Neural Network's perfect recall is a remarkable achievement, the Random Forest offers superior overall accuracy (99.26%), a perfect precision score that ensures zero wasted computational overhead, and critical model transparency. By utilizing the Random Forest, administrators can trust both what the model predicts and why it makes those predictions, allowing for highly efficient, trigger-based dynamic routing in real-world IoT environments.

Figure 3*ROC Curve*

While our Confusion Matrix shows our exact predictions about the neural network model, this ROC curve with a near-perfect AUC score of 0.9983 proves that our Neural Network has an outstanding ability to clearly distinguish between effective and non-effective optimization scenarios, rather than just getting lucky.

Figure 4*Multi Model Comparison*

We ultimately selected the Neural Network because its Recall score on the bar chart hit a perfect 100%, outperforming the Random Forest in catching every single energy-saving opportunity.

Figure*Confusion Matrix*

The generated confusion matrix visually validates our primary business objective: zero False Negatives. By catching all 68 out of 68 true 'Effective' scenarios, the matrix proves that deploying this Neural Network guarantees the IoT system will never miss an opportunity to reduce energy consumption.

True Positives (68 out of 68): The model correctly predicted 68 networks where optimization was effective. (This is your perfect 100% Recall).

False Negatives (0 out of 68): The model missed ZERO effective scenarios. It caught every single one.

True Negatives (~63 out of 68): The model correctly identified about 63 networks where optimization would not help, successfully saving the network from wasting computational power.

False Positives (~5 out of 68): The model made a minor mistake about 5 times, predicting that the optimization would be effective when it actually wasn't. (This represents your 93% Precision).

8. Conclusion & Business Implications

The primary objective of this project was to develop a predictive machine learning pipeline capable of determining the operational effectiveness of the ICSHSO dynamic routing algorithm in IoT sensor networks. Through the implementation of a robust Medallion data architecture, we successfully ingested, cleaned, and engineered 500 simulated network scenarios. By applying SMOTE data augmentation to resolve inherent class imbalances, we provided a mathematically stable environment for model training, resulting in exceptional predictive performance across all evaluated architectures.

While the Deep Neural Network achieved a flawless 100% Recall, the Random Forest Classifier emerged as the optimal deployment choice for this IoT energy management system. Achieving an outstanding overall accuracy of 99.26%, an F1-Score of 0.9926, and a perfect 100% Precision, the Random Forest guarantees that computational resources are never wasted on sub-optimal network configurations.

Furthermore, the Random Forest model provided critical "white-box" explainability. By extracting the model's feature importances, we determined that the effectiveness of ICSHSO

routing is not random; it is heavily dictated by the network's Packet Delivery Ratio (PDR%) and its Network Lifetime Improvement metrics.

Final Recommendation for IoT Deployment Based on these findings, we recommend that network administrators abandon a "one-size-fits-all" approach to IoT routing. Instead, the Random Forest model should be deployed as a real-time trigger mechanism. The ICSHSO optimization algorithm should remain dormant to conserve baseline energy, and be dynamically activated by the AI only when the model detects specific, high-stress network states—particularly those struggling with packet delivery or rapid battery depletion. This predictive, trigger-based approach ensures maximum energy conservation, extends the physical lifespan of the sensor nodes, and drastically reduces the overhead costs associated with IoT network maintenance.